SE444: Introduction to Artificial Intelligence
Final Exam — Solutions — Fall 2025

Prof. Anis Koubaa
akoubaa@alfaisal.edu
Alfaisal University

Date: Monday, December 22, 2025 Time: 12:30 PM – 2:30 PM (120 min)

Instructions

- This is a closed-book exam. The University's code of ethics applies.
- Answer each exam question to the best of your knowledge.
- Clearly state any assumptions made that might be needed to understand your solution.
- Show your work – any sign of serious effort will be considered.
- No marks will be given for not making any effort.

Student Information

Student ID:	Student Name:

Summary of Exam Questions

Part	Topic	Time	Score	Mark	CLOs
Part I MCQ	BN Structure / d-Separation	5 min	1		CLO 1
	Bayes' Rule Application	5 min	1		CLO 1
	Decision Theory / MDPs	5 min	1		CLO 1, 2
	Game Theory — Nash Equilibrium	5 min	1		CLO 1, 3
	Classical Planning (STRIPS)	5 min	1		CLO 1, 2
	Project Code — Logic Agent	5 min	1		CLO 2
Part II Robot Nav	BN Robot Perception	15 min	4		CLO 1
	Multi-Sensor Fusion	15 min	4		CLO 1, 2
	Path Decision (MEU)	15 min	5		CLO 1, 2
	Project Code (Bayes)	15 min	3		CLO 2
Part III Multi-Robot	STRIPS Planning	12 min	3		CLO 1, 2
	Coordination Game	10 min	3		CLO 1, 3
	Project: Logic Agent	8 min	2		CLO 2
Total for Part I (MCQ)		30 min	6		
Total for Part II (Robot Navigation)		60 min	16		
Total for Part III (Multi-Robot)		30 min	8		
Total Points		120 min	**30**		

CLOs: 1 = Knowledge & Understanding, 2 = Skills (Implementation), 3 = Values & Ethics

Part I. Critical-Thinking Multiple Choice Questions

Total Score: 6 points **Total Time:** 30 minutes

Instructions: Each question may have **one or more correct answers**. Select **all that apply**. Partial credit will be awarded for partially correct answers.

Answer Key Summary

Option	Q1	Q2	Q3	Q4	Q5	Q6
A						
B		**X**	**X**		**X**	**X**
C						
D	**X**					
E		**X**		**X**		

Correct Answers:

- **Q1 (d-Separation):** D
- **Q2 (Bayes):** B, E
- **Q3 (MDPs):** B
- **Q4 (Game Theory):** E
- **Q5 (Planning):** B
- **Q6 (Code):** B

Question I.1: Bayesian Network Structure & d-Separation

Time: 5 min **Score:** 1 pt **CLOs:** CLO 1

Consider the Bayesian Network: $A \to C \leftarrow B$, $A \to D \leftarrow E$, $C \to F$
Which conditional independence statement is TRUE?

A) $A \perp B \mid C$

B) $A \perp E \mid D$

C) $A \perp B \mid F$

D) $B \perp E$

E) $A \perp F$

Answer: D

Network Structure:

- C is a collider between A and B (v-structure: $A \to C \leftarrow B$)
- D is a collider between A and E (v-structure: $A \to D \leftarrow E$)
- F is a descendant of collider C (chain: $C \to F$)

Explanation:

A) **Incorrect.** When C is observed, the collider $A \to C \leftarrow B$ becomes "open," making A and B **dependent** (explaining away effect).

B) **Incorrect.** When D is observed, the collider $A \to D \leftarrow E$ becomes "open," making A and E **dependent**.

C) **Incorrect.** F is a descendant of collider C. Observing a descendant of a collider also "opens" the collider. Therefore, conditioning on F opens the path $A \to C \leftarrow B$, making A and B **dependent**.

D) **Correct.** The only path between B and E is: $B \to C \leftarrow A \to D \leftarrow E$. This path contains **two unobserved colliders**: C and D. An unobserved collider blocks a path. Since both colliders are unobserved, the path is **blocked**. Therefore, $B \perp E$.

E) **Incorrect.** Path $A \to C \to F$ is a chain. With C unobserved, information flows from A to F. So A and F are **dependent**.

Key Concept: Colliders block paths when unobserved but open paths when observed (including when descendants are observed). Multiple colliders on a path means multiple blocking points.

Question I.2: Bayes' Rule Application

Time: 5 min **Score:** 1 pt **CLOs:** CLO 1

A forensic DNA analysis in a criminal investigation: $P(\text{Guilty}) = 0.005$ (1 in 200 suspects), $P(\text{Match}|\text{Guilty}) = 0.99$, $P(\text{Match}|\neg\text{Guilty}) = 0.03$ (false positive). A DNA match is found. Which statement is TRUE?

A) $P(\text{Guilty}|\text{Match}) > 0.5$

B) $P(\text{Guilty}|\text{Match})$ is between 0.10 and 0.20

C) $P(\text{Guilty}|\text{Match}) < 0.05$

D) If $P(\text{Guilty})$ increases to 0.20, then $P(\text{Guilty}|\text{Match}) > 0.95$

E) $P(\text{Guilty}|\text{Match}) < P(\text{Match}|\text{Guilty})$

Answer: B, C

Calculation:

$$P(G|M) = \frac{P(M|G) \cdot P(G)}{P(M)}$$

$$P(M) = P(M|G)P(G) + P(M|\neg G)P(\neg G)$$

$$= 0.99 \times 0.005 + 0.03 \times 0.995 = 0.00495 + 0.02985 = 0.0348$$

$$P(G|M) = \frac{0.99 \times 0.005}{0.0348} = \frac{0.00495}{0.0348} \approx 0.142 \text{ (about 14.2\%)}$$

Analysis:

A) **Incorrect.** $P(G|M) \approx 0.142 < 0.5$. Despite the DNA match, there's only a 14% chance of guilt.

B) **Correct.** $P(G|M) \approx 0.142$, which is between 0.10 and 0.20.

C) **Incorrect.** $P(G|M) \approx 0.142 > 0.05$.

D) **Incorrect.** With $P(G) = 0.20$:

$$P(M) = 0.99 \times 0.20 + 0.03 \times 0.80 = 0.198 + 0.024 = 0.222$$

$$P(G|M) = \frac{0.198}{0.222} \approx 0.892 < 0.95$$

E) **Correct.** $P(G|M) \approx 0.142 < 0.99 = P(M|G)$. The posterior is much smaller than the likelihood due to the low prior.

Key Concept: Base rate neglect — even with accurate forensic evidence (99% sensitivity), the posterior probability can be surprisingly low when the prior is small. Students must compute the exact value to identify the correct range.

Question I.3: Decision Theory & MDPs

Time: 5 min **Score:** 1 pt **CLOs:** CLO 1, 2

An autonomous robot uses an MDP with discount factor $\gamma = 0.8$ for navigation. At state s, the robot has two actions:

- **Action A:** Immediate reward $R = 10$, transitions to a terminal state
- **Action B:** Immediate reward $R = 2$, transitions to state s' where $V(s') = 15$

Which statement is TRUE?

A) Action A is optimal at state s

B) $V(s) = 10$ if the robot always chooses Action A

C) $V(s) = 10$ under the optimal policy

D) $Q(s, A) > Q(s, B)$

E) Changing γ to 0.5 would not affect which action is optimal

Answer: B

Calculation:

$$Q(s, A) = R_A + \gamma \cdot V(\text{terminal}) = 10 + 0.8 \times 0 = 10$$
$$Q(s, B) = R_B + \gamma \cdot V(s') = 2 + 0.8 \times 15 = 2 + 12 = 14$$
$$V(s) = \max\{Q(s, A), Q(s, B)\} = \max\{10, 14\} = 14$$

Analysis:

A) **Incorrect.** Action B is optimal since $Q(s, B) = 14 > Q(s, A) = 10$.

B) **Correct.** If the robot always chooses Action A (a fixed, non-optimal policy), then $V(s) = Q(s, A) = 10$. This is the value of state s under that specific policy.

C) **Incorrect.** Under the optimal policy, the robot chooses Action B, so $V(s) = 14$, not 10.

D) **Incorrect.** $Q(s, A) = 10 < 14 = Q(s, B)$.

E) **Incorrect.** With $\gamma = 0.5$:

$$Q(s, A) = 10$$
$$Q(s, B) = 2 + 0.5 \times 15 = 2 + 7.5 = 9.5$$

Now $Q(s, A) = 10 > 9.5 = Q(s, B)$, so Action A becomes optimal. The discount factor **does** affect the optimal action.

Key Concept: The discount factor γ determines how much the agent values future rewards. Lower γ favors immediate rewards, while higher γ makes delayed rewards more attractive.

Question I.4: Game Theory — Nash Equilibrium

Time: 5 min **Score:** 1 pt **CLOs:** CLO 1, 3

Consider the game (payoffs Row, Col):

	Left	**Right**
Up	(2, 1)	(1, 2)
Down	(3, 0)	(0, 3)

Which statement is TRUE?

A) Row player has a strictly dominant strategy

B) (Up, Left) is a Nash Equilibrium

C) (Down, Right) is a Nash Equilibrium

D) (Down, Left) is a Nash Equilibrium

E) Column player has a strictly dominant strategy

Answer: E

Analysis:

Step 1: Check for Dominant Strategies

Row player (Up vs Down):

- Against Left: Down(3) > Up(2) ⇒ Down is better
- Against Right: Up(1) > Down(0) ⇒ Up is better

Neither strategy dominates. Row has **no dominant strategy**.

Column player (Left vs Right):

- Against Up: Right(2) > Left(1) ⇒ Right is better
- Against Down: Right(3) > Left(0) ⇒ Right is better

Right is better in **both** cases. Column has a **strictly dominant strategy: Right**.

Step 2: Check Nash Equilibria

- **(Up, Left):** Row gets 2. Switching to Down gives 3 > 2. Row wants to switch. **Not NE.**
- **(Up, Right):** Row gets 1. Switching to Down gives 0. Row doesn't switch. Column gets 2. Switching to Left gives 1. Column doesn't switch. **This IS a NE!**
- **(Down, Left):** Column gets 0. Switching to Right gives 3 > 0. Column wants to switch. **Not NE.**
- **(Down, Right):** Row gets 0. Switching to Up gives 1 > 0. Row wants to switch. **Not NE.**

The unique pure-strategy NE is **(Up, Right)**, not listed in options B, C, or D.

Evaluation of Options:

A) **Incorrect.** Row has no dominant strategy (shown above).

B) **Incorrect.** (Up, Left) is not a NE.

C) **Incorrect.** (Down, Right) is not a NE.

D) **Incorrect.** (Down, Left) is not a NE.

E) **Correct.** Column's Right strictly dominates Left.

Key Concept: A strictly dominant strategy yields a higher payoff regardless of the opponent's choice. When only one player has a dominant strategy, use it to predict behavior and find equilibria.

Question I.5: Classical Planning — STRIPS

Time: 5 min **Score:** 1 pt **CLOs:** CLO 1, 2

Consider the following STRIPS action schema for UAV navigation:

```
Action: FlyTo(uav, from, to)
  Precond: At(uav, from), HasBattery(uav), Cleared(to)
  Add:     At(uav, to), Visited(to)
  Delete:  At(uav, from), Cleared(to)
```

Given initial state: `At(UAV1, Base), HasBattery(UAV1), Cleared(WP1), Cleared(WP2)`
After executing `FlyTo(UAV1, Base, WP1)`, which statement is TRUE?

A) `At(UAV1, Base)` is still true

B) `Visited(WP1)` is true

C) `Cleared(WP1)` is still true

D) `HasBattery(UAV1)` becomes false

E) The action cannot execute because `Visited(WP1)` is not in preconditions

Answer: B

Step 1: Check Preconditions

For `FlyTo(UAV1, Base, WP1)` to execute, we need:

- `At(UAV1, Base)` — ✓Present in initial state
- `HasBattery(UAV1)` — ✓Present in initial state
- `Cleared(WP1)` — ✓Present in initial state

All preconditions are satisfied. The action **can execute**.

Step 2: Apply Effects

Add List: `At(UAV1, WP1), Visited(WP1)`
Delete List: `At(UAV1, Base), Cleared(WP1)`
Resulting State:

- `At(UAV1, WP1)` — Added
- `Visited(WP1)` — Added
- `HasBattery(UAV1)` — Unchanged (not in Delete list)
- `Cleared(WP2)` — Unchanged
- `At(UAV1, Base)` — **Deleted**
- `Cleared(WP1)` — **Deleted**

Evaluation of Options:

A) **Incorrect.** `At(UAV1, Base)` is in the Delete list, so it becomes **false**.

B) **Correct.** `Visited(WP1)` is in the Add list, so it becomes **true**.

C) **Incorrect.** `Cleared(WP1)` is in the Delete list, so it becomes **false**.

D) **Incorrect.** `HasBattery(UAV1)` is **not** in the Delete list, so it remains true.

E) **Incorrect.** `Visited(WP1)` does not need to be a precondition — it's an **effect**. The action executes successfully.

Key Concept: STRIPS effects: Add list facts become true, Delete list facts become false, and all other facts remain unchanged (frame axiom).

Question I.6: Project Code — Logic Agent

Time: 5 min **Score:** 1 pt **CLOs:** CLO 2

```
def get_action(self, percept):
    self.kb.tell(make_percept_sentence(percept, self.time))
    action = None
    for a in self.possible_actions:
        if self.kb.ask(is_safe_action(a, self.time)):
            action = a
            break
    self.kb.tell(make_action_sentence(action, self.time))
    self.time += 1
    return action
```

Which statement is TRUE about this agent code?

A) This is a model-based reflex agent

B) This is a knowledge-based (logic) agent

C) `kb.ask()` modifies the knowledge base

D) The agent uses probability distributions to select actions

E) `self.time` only tracks visited grid cells

Answer: B

Analysis:

The code shows a classic **knowledge-based agent** architecture:

1. `kb.tell()` adds new knowledge (percepts, actions) to the knowledge base
2. `kb.ask()` queries the KB using logical inference (without modifying it)
3. Actions are selected based on logical entailment (`is_safe_action`)
4. `self.time` is a logical time step for temporal reasoning

Evaluation of Options:

A) **Incorrect.** Model-based reflex agents use an internal state but not a full knowledge base with logical inference. This agent uses `kb.tell()` and `kb.ask()`, which are characteristic of knowledge-based agents.

B) **Correct.** The agent maintains a knowledge base (`self.kb`), uses `tell()` to add knowledge, and `ask()` for logical queries. This is the defining pattern of a knowledge-based (logic) agent.

C) **Incorrect.** `kb.ask()` performs a **query** (inference) and returns a result without modifying the KB. Only `kb.tell()` modifies the knowledge base.

D) **Incorrect.** There are no probability distributions, Bayesian updates, or stochastic elements. The agent uses deterministic logical inference.

E) **Incorrect.** `self.time` is a logical time step used for temporal reasoning in the KB (e.g., "action A was taken at time 5"). It does not track spatial positions or visited cells.

Key Concept: Knowledge-based agents use a knowledge base (KB) with `tell()` to add knowledge and `ask()` to query via logical inference, enabling reasoning about complex environments.

Part II. Robot Navigation Under Uncertainty

Total Score: 16 points **Total Time:** 60 minutes

Question II.1: BN for Robot Perception (4 pts, 15 min)

Scenario: A navigation robot uses multiple sensors to detect obstacles:

- `Obstacle (O)`: True obstacle present at location
- `Camera (C)`: Camera detects obstacle
- `Lidar (L)`: Lidar detects obstacle
- `Weather (W)`: Bad weather conditions
- `PathBlocked (P)`: Robot path is blocked
- `Alert (A)`: System generates navigation alert

Dependencies: O causes both C and L readings. W affects C accuracy. P depends on O and W. A depends on P.

Tasks:

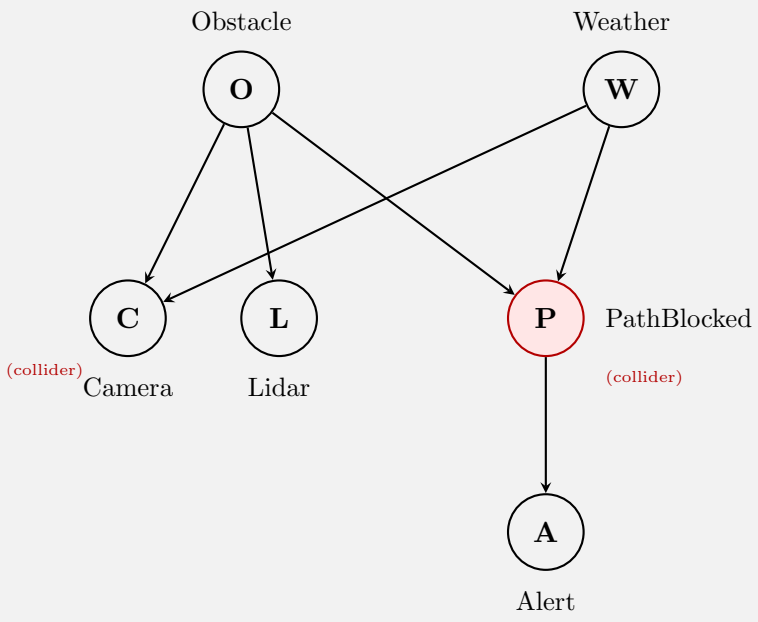
(a) [2 pts] Draw the Bayesian Network structure with all 6 nodes.

(b) [1 pt] Are $L \perp A \mid P$? Use d-separation. Trace all paths.

(c) [1 pt] Identify ALL collider nodes and explain "explaining away" for one.

(a) Bayesian Network Structure [2 pts]:

The 6-Node Robot Perception Network:



Summary of Edges:

Edge	Meaning
$O \to C$	Obstacle causes camera detection
$O \to L$	Obstacle causes lidar detection
$W \to C$	Weather affects camera accuracy
$O \to P$, $W \to P$	Both O and W cause path blocked (P is collider)
$P \to A$	Path blocked triggers alert

Grading: 1pt for correct 6-node structure, 1pt for correct edge directions.

(b) Is $L \perp A \mid P$? [1 pt]:

Find all paths from L to A:

1. **Path 1:** $L \leftarrow O \to P \to A$
2. **Path 2:** $L \leftarrow O \to P \leftarrow W \to C$ (doesn't reach A)

The only path from L to A is: $L \leftarrow O \to P \to A$
Check the path given P is observed:
Path: $L \leftarrow O \to P \to A$

- At node O: This is a **fork** ($L \leftarrow O \to P$)
- O is NOT observed
- Normally a fork is open... but we must continue checking
- At node P: This is a **chain** ($O \to P \to A$)
- P IS observed, so the chain is **BLOCKED**

Conclusion: The path is blocked at P.

YES, $L \perp A \mid P$ (L and A are conditionally independent given P)

Intuition: Once we know whether the path is blocked (P), the lidar reading (L) tells us nothing new about whether an alert is generated (A), because A only depends on P.
Grading: 1pt for correct answer with path tracing.

(c) Identify ALL Collider Nodes and Explaining Away [1 pt]:

What is a Collider?
A **collider** is a node that has two or more parent nodes pointing into it.
All Colliders in this 6-Node Network:

1. **C (Camera):** $O \to C \leftarrow W$ — both Obstacle and Weather affect camera
2. **P (PathBlocked):** $O \to P \leftarrow W$ — both Obstacle and Weather cause path blocked

Note: L is NOT a collider (only one parent: O). A is NOT a collider (only one parent: P).

Explaining Away Effect (Example with P):

1. **Before observing P:** Obstacle (O) and Weather (W) are independent.
2. **After observing P = True (path is blocked):** O and W become dependent!
    - The path being blocked must have a cause: either obstacle OR bad weather (or both)
    - If we learn W = True (bad weather), this **explains** the blocked path
    - This makes obstacle **less likely** — we have an alternative explanation!

$$P(O = T \mid P = \text{True}, W = T) < P(O = T \mid P = \text{True})$$

Key Takeaway: Observing a collider "opens" the path between its parents, creating a dependency where none existed before.

Grading: 0.5pt for identifying colliders (C and P), 0.5pt for explaining effect.

Question II.2: Multi-Sensor Fusion (4 pts, 15 min)

Scenario: A robot uses TWO sensors sequentially to update its belief about an obstacle.

Prior: $P(\text{Obstacle} = T) = 0.2$

Sensor Accuracies:

- **Camera:** $P(\text{Detect}|\text{Obstacle} = T) = 0.8$, $P(\text{Detect}|\text{Obstacle} = F) = 0.15$
- **Lidar:** $P(\text{Detect}|\text{Obstacle} = T) = 0.9$, $P(\text{Detect}|\text{Obstacle} = F) = 0.1$

Observations: Camera detects obstacle (1st), then Lidar detects obstacle (2nd).

Tasks:

(a) [2 pts] After Camera=Detect, compute $P(\text{Obstacle} = T \mid \text{Camera} = \text{Detect})$. This becomes the new prior. *Write one sentence interpreting the result.*

(b) [2 pts] After Lidar=Detect, compute $P(\text{Obstacle} = T \mid \text{Camera} = \text{Detect}, \text{Lidar} = \text{Detect})$ using the posterior from (a) as the prior. *Write one sentence interpreting the result.*

(a) First Sensor Update: Camera Detection [2 pts]:

Goal: Compute $P(\text{Obstacle} = T \mid \text{Camera} = \text{Detect})$

Given:

- Prior: $P(O = T) = 0.2$, so $P(O = F) = 0.8$
- Likelihood: $P(\text{Cam} = D \mid O = T) = 0.8$
- False positive: $P(\text{Cam} = D \mid O = F) = 0.15$

Step 1: Apply Bayes' Rule

$$P(O = T \mid \text{Cam} = D) = \frac{P(\text{Cam} = D \mid O = T) \cdot P(O = T)}{P(\text{Cam} = D)}$$

Step 2: Compute $P(\text{Cam} = D)$ using Total Probability

$$\begin{aligned} P(\text{Cam} = D) &= P(\text{Cam} = D \mid O = T) \cdot P(O = T) + P(\text{Cam} = D \mid O = F) \cdot P(O = F) \\ &= 0.8 \times 0.2 + 0.15 \times 0.8 \\ &= 0.16 + 0.12 = 0.28 \end{aligned}$$

Step 3: Compute Posterior

$$P(O = T \mid \text{Cam} = D) = \frac{0.8 \times 0.2}{0.28} = \frac{0.16}{0.28} = \boxed{\frac{4}{7} \approx 0.571}$$

Interpretation: After the camera detects an obstacle, our belief in an obstacle increases from 20% to 57.1%.

Grading: 1pt for correct Bayes' setup, 1pt for correct calculation.

(b) Second Sensor Update: Lidar Detection [2 pts]:
Goal: Compute $P(O = T \mid \text{Cam} = D, \text{Lidar} = D)$
Key Insight: The posterior from part (a) becomes our NEW prior!

$$P_{\text{new}}(O = T) = \frac{4}{7} \approx 0.571, \quad P_{\text{new}}(O = F) = \frac{3}{7} \approx 0.429$$

Lidar Sensor Data:

- $P(\text{Lidar} = D \mid O = T) = 0.9$
- $P(\text{Lidar} = D \mid O = F) = 0.1$

Step 1: Apply Bayes' Rule (second update)

$$P(O = T \mid \text{Cam} = D, \text{Lidar} = D) = \frac{P(\text{Lidar} = D \mid O = T) \cdot P_{\text{new}}(O = T)}{P(\text{Lidar} = D)}$$

Step 2: Compute $P(\textbf{Lidar} = D)$ with new prior

$$\begin{aligned} P(\text{Lidar} = D) &= P(\text{Lidar} = D \mid O = T) \cdot P_{\text{new}}(O = T) + P(\text{Lidar} = D \mid O = F) \cdot P_{\text{new}}(O = F) \\ &= 0.9 \times \frac{4}{7} + 0.1 \times \frac{3}{7} \\ &= \frac{3.6}{7} + \frac{0.3}{7} = \frac{3.9}{7} \approx 0.557 \end{aligned}$$

Step 3: Compute Final Posterior

$$\begin{aligned} P(O = T \mid \text{Cam} = D, \text{Lidar} = D) &= \frac{0.9 \times \frac{4}{7}}{\frac{3.9}{7}} = \frac{\frac{3.6}{7}}{\frac{3.9}{7}} = \frac{3.6}{3.9} \\ &= \frac{36}{39} = \frac{12}{13} \approx \boxed{0.923} \end{aligned}$$

Summary of Sequential Belief Updates:

Stage	P(Obstacle=T)	What happened
Initial Prior	0.200	Base rate (1 in 5)
After Camera=Detect	0.571	Increased by 2.86×
After Lidar=Detect	0.923	Increased to 92.3%

Key Takeaway: Sequential sensor fusion multiplicatively combines evidence. Each sensor independently increases our confidence, demonstrating how Bayesian inference handles cumulative information.
Grading: 1pt for using part (a) as new prior, 1pt for correct final calculation.

Question II.3: Sensor-Based Navigation Decision (5 pts, 15 min)

Scenario: A robot uses a proximity sensor before deciding to cross a corridor. The sensor reading depends on whether an obstacle is actually present.
Prior: $P(\text{Obstacle} = T) = 0.3$
Sensor CPT:

Sensor Reading	Obstacle=T	Obstacle=F
Alert	0.85	0.10
Clear	0.15	0.90

Utility Table (robot's payoff):

Action	Obstacle=T	Obstacle=F
Cross	−50	+80
Wait	+10	+10

Tasks:

(a) [2 pts] The sensor reads **Alert**. Compute $P(\text{Obstacle} = T \mid \text{Sensor} = \text{Alert})$. Show all steps.

(b) [3 pts] Given Sensor=Alert, compute the **Expected Utility (EU)** for each action and determine the **MEU-optimal** action. Explain why this action is rational.

(a) Compute $P(\textbf{Obstacle} = T \mid \textbf{Sensor} = \textbf{Alert})$ [2 pts]:

Given:

- Prior: $P(O = T) = 0.3$, $P(O = F) = 0.7$
- From CPT: $P(\text{Alert} \mid O = T) = 0.85$, $P(\text{Alert} \mid O = F) = 0.10$

Step 1: Apply Bayes' Rule

$$P(O = T \mid \text{Alert}) = \frac{P(\text{Alert} \mid O = T) \cdot P(O = T)}{P(\text{Alert})}$$

Step 2: Compute $P(\textbf{Alert})$ using Total Probability

$$\begin{aligned} P(\text{Alert}) &= P(\text{Alert} \mid O = T) \cdot P(O = T) + P(\text{Alert} \mid O = F) \cdot P(O = F) \\ &= 0.85 \times 0.3 + 0.10 \times 0.7 \\ &= 0.255 + 0.07 = 0.325 \end{aligned}$$

Step 3: Compute Posterior

$$P(O = T \mid \text{Alert}) = \frac{0.85 \times 0.3}{0.325} = \frac{0.255}{0.325} \approx \boxed{0.785}$$

Interpretation: When the sensor alerts, there's a 78.5% probability an obstacle is actually present (up from 30% prior).

Grading: 1pt for correct setup, 1pt for correct calculation.

(b) Expected Utility and MEU-optimal action [3 pts]:

Using the posterior from (a) as our new belief state: $P(O = T \mid \text{Alert}) \approx 0.785$ and $P(O = F \mid \text{Alert}) \approx 0.215$.

Logic: Rational decisions must be based on the *current* belief. Since we have observed an Alert, we calculate Expected Utility conditioned on that observation.

EU for Cross:

$$\begin{aligned} EU(\text{Cross} \mid \text{Alert}) &= P(O = T \mid \text{Alert}) \times U(\text{Cross}, O = T) + P(O = F \mid \text{Alert}) \times U(\text{Cross}, O = F) \\ &= 0.785 \times (-50) + 0.215 \times 80 \\ &= -39.25 + 17.2 = \boxed{-22.05} \end{aligned}$$

EU for Wait:

$$\begin{aligned} EU(\text{Wait} \mid \text{Alert}) &= P(O = T \mid \text{Alert}) \times U(\text{Wait}, O = T) + P(O = F \mid \text{Alert}) \times U(\text{Wait}, O = F) \\ &= 0.785 \times 10 + 0.215 \times 10 \\ &= 7.85 + 2.15 = \boxed{10} \end{aligned}$$

MEU Decision: Since $EU(\text{Wait} \mid \text{Alert}) = 10$ is significantly greater than $EU(\text{Cross} \mid \text{Alert}) = -22.05$:

MEU-optimal: Wait

Interpretation: When the sensor alerts, the high probability of obstacle (78.5
Summary Table:

Action	EU	Decision
Cross	−22.05	
Wait	+10	✓MEU

Logical Reasoning: Despite the tempting +80 utility for a safe crossing, the **risk-adjusted** utility of crossing is negative. The 78.5% probability of a collision (costing -50) creates an expected loss that makes "Wait" (+10 guaranteed) the only rational choice. This demonstrates how even a 10% false alarm rate ($P(Alert|O = F) = 0.1$) is low enough to make the Alert highly actionable.

Grading: 1.5pt for correct EU calculations (using posterior), 1.5pt for correct MEU decision and reasoning.

Question II.4: Project Code Analysis — Bayesian Reasoning (3 pts, 15 min)

Given: The following code from the RoboMind project's `bayes_reasoning.py`:

```python
def bayes_update(prior, likelihood, evidence):
    """
    Update belief using Bayes' Rule.
    P(H|E) = P(E|H) * P(H) / P(E)
    """
    if evidence == 0:
        return prior  # Avoid division by zero
    posterior = (likelihood * prior) / evidence
    return posterior

def compute_evidence(prior, likelihood_h, likelihood_not_h):
    """
    P(E) = P(E|H)*P(H) + P(E|~H)*P(~H)
    """
    return (likelihood_h * prior) + (likelihood_not_h * (1 - prior))
```

Scenario: The robot's sensor has the following characteristics:

- $P(\text{Detect} \mid \text{Obstacle}) = 0.9$ (True Positive Rate)
- $P(\text{Detect} \mid \neg\text{Obstacle}) = 0.15$ (False Positive Rate)

The robot's prior belief that a cell contains an obstacle is $P(\text{Obstacle}) = 0.4$.
Tasks:

(a) [1 pt] The sensor reports **Detect**. Trace through the `compute_evidence()` function call. Show the values of each variable and the return value.

(b) [1 pt] Trace through the `bayes_update()` function to compute the posterior probability. What is $P(\text{Obstacle} \mid \text{Detect})$?

(c) [1 pt] If the code used `evidence = 0.5` instead of computing it properly, what would the posterior be? Why is computing evidence correctly important?

(a) Trace `compute_evidence()` [1 pt]:
Function call:

```
compute_evidence(prior=0.4, likelihood_h=0.9, likelihood_not_h=0.15)
```

Step-by-step trace:

- `prior` = 0.4
- `likelihood_h` = 0.9 (P(Detect | Obstacle))
- `likelihood_not_h` = 0.15 (P(Detect | ¬Obstacle))
- `1 - prior` = 0.6

Calculation:

$$\begin{aligned}\text{evidence} &= (0.9 \times 0.4) + (0.15 \times 0.6) \\ &= 0.36 + 0.09 = \boxed{0.45}\end{aligned}$$

Return value: `0.45`
Grading: 1pt for correct trace with values shown.

(b) Trace `bayes_update()` [1 pt]:
Function call:

```
bayes_update(prior=0.4, likelihood=0.9, evidence=0.45)
```

Step-by-step trace:

- Check: `evidence == 0`? No ($0.45 \neq 0$)
- `prior` = 0.4
- `likelihood` = 0.9
- `evidence` = 0.45

Calculation:

$$\begin{aligned}\text{posterior} &= \frac{\text{likelihood} \times \text{prior}}{\text{evidence}} \\ &= \frac{0.9 \times 0.4}{0.45} \\ &= \frac{0.36}{0.45} = \boxed{0.8}\end{aligned}$$

Return value: `0.8`

$$P(\text{Obstacle} \mid \text{Detect}) = 80\%$$

Interpretation: The sensor detection increased the belief from 40% to 80%.
Grading: 1pt for correct trace with final answer.

(c) Wrong evidence value [1 pt]:
If `evidence = 0.5` (incorrect):

$$\text{posterior} = \frac{0.9 \times 0.4}{0.5} = \frac{0.36}{0.5} = \boxed{0.72}$$

Problem: Using 0.5 instead of 0.45 gives 72% instead of 80%.

Why computing evidence correctly matters:

- **Evidence is the normalizing constant** that ensures probabilities are properly scaled
- Wrong evidence = wrong posterior probability = wrong decisions
- The robot might incorrectly proceed into a dangerous cell or avoid a safe one
- Bayesian inference requires accurate total probability P(E) to work correctly
- In the project, this could cause the agent to make suboptimal navigation choices

Key insight: The evidence term accounts for ALL ways the observation could occur. Ignoring this leads to biased probability estimates.

Grading: 0.5pt for correct calculation, 0.5pt for explanation.

Part III. Multi-Robot Cooperation

Total Score: 8 points **Total Time:** 30 minutes

Question III.1: Robot Task Planning — STRIPS (3 pts, 12 min)

Rescue Robot Scenario: A rescue robot must pick up a victim and deliver them to a safe zone.
Predicates: `At(robot,loc)`, `Holding(robot,victim)`, `VictimAt(victim,loc)`, `Safe(loc)`
Initial State: At(R1, Base), VictimAt(V1, Zone3), Safe(Zone1), ¬Holding(R1, V1)
Goal State: VictimAt(V1, Zone1)
Tasks:

(a) [1 pt] Define STRIPS action `Pickup(robot, victim, loc)`: Preconditions, Add List, Delete List.

(b) [2 pts] Show a forward search plan to achieve the goal. Include actions: MoveTo, Pickup, PutDown.

(a) STRIPS Action `Pickup(robot, victim, loc)` [1 pt]:
Preconditions:

- `At(robot, loc)` — Robot must be at the victim's location
- `VictimAt(victim, loc)` — Victim must be at that location
- ¬`Holding(robot, victim)` — Robot not already holding victim

Add List (Effects that become true):

- `Holding(robot, victim)` — Robot now holding victim

Delete List (Effects that become false):

- `VictimAt(victim, loc)` — Victim no longer at that location (being carried)

Grading: 0.5pt for correct preconditions, 0.25pt for add list, 0.25pt for delete list.

(b) Forward Search Plan [2 pts]:
Initial State:

`At(R1, Base), VictimAt(V1, Zone3), Safe(Zone1), ~Holding(R1, V1)`

Goal State: `VictimAt(V1, Zone1)`

Step 1: MoveTo(R1, Base, Zone3)

- Precond: `At(R1, Base)` — satisfied
- Add: `At(R1, Zone3)`
- Delete: `At(R1, Base)`
- State: `At(R1, Zone3), VictimAt(V1, Zone3), Safe(Zone1)`

Step 2: Pickup(R1, V1, Zone3)

- Precond: `At(R1, Zone3), VictimAt(V1, Zone3)` — all satisfied
- Add: `Holding(R1, V1)`
- Delete: `VictimAt(V1, Zone3)`
- State: `At(R1, Zone3), Holding(R1, V1), Safe(Zone1)`

Step 3: MoveTo(R1, Zone3, Zone1)

- Precond: `At(R1, Zone3)` — satisfied
- Add: `At(R1, Zone1)`
- Delete: `At(R1, Zone3)`
- State: `At(R1, Zone1), Holding(R1, V1), Safe(Zone1)`

Step 4: PutDown(R1, V1, Zone1)

- Precond: `At(R1, Zone1), Holding(R1, V1)` — all satisfied
- Add: `VictimAt(V1, Zone1)`
- Delete: `Holding(R1, V1)`
- **Final State:** `At(R1, Zone1), VictimAt(V1, Zone1)` — **GOAL ACHIEVED ✓**

Plan: MoveTo(R1, Base, Zone3) → Pickup(R1, V1, Zone3) → MoveTo(R1, Zone3, Zone1) → PutDown(R1, V1, Zone1)
Grading: 1pt for correct sequence, 1pt for showing state changes.

Question III.2: Robot Coordination Game (3 pts, 10 min)

Scenario: Two delivery robots (R1 and R2) approach an intersection. Each must choose a lane (Left or Right) to avoid collision.

R1 \ R2	Left	Right
Left	(0, 0)	(5, 5)
Right	(5, 5)	(0, 0)

Tasks:

(a) [1 pt] Identify any strictly dominated strategies.

(b) [2 pts] Find all pure-strategy Nash Equilibria. Mark best responses.

(a) Dominated Strategies [1 pt]:
Check for R1 (Row Player):

- Left gives payoffs: (0, 5) depending on R2's choice
- Right gives payoffs: (5, 0) depending on R2's choice
- Left is better when R2 plays Right (5 ¿ 0)
- Right is better when R2 plays Left (5 ¿ 0)
- **Neither strategy strictly dominates the other**

Check for R2 (Column Player):

- Same logic applies by symmetry
- **No dominated strategies for R2 either**

No strictly dominated strategies exist

This is a coordination game — success depends on matching/mismatching choices.
Grading: 1pt for correct analysis showing no dominated strategies.

(b) Pure-Strategy Nash Equilibria [2 pts]:
Method: Find best responses for each player.
R1's Best Response (underline R1's best payoff in each column):

- If R2 plays Left: R1's payoffs are 0 (Left), 5 (Right). Best: Right (5)
- If R2 plays Right: R1's payoffs are 5 (Left), 0 (Right). Best: Left (5)

R2's Best Response (underline R2's best payoff in each row):

- If R1 plays Left: R2's payoffs are 0 (Left), 5 (Right). Best: Right (5)
- If R1 plays Right: R2's payoffs are 5 (Left), 0 (Right). Best: Left (5)

Finding Nash Equilibria (cells where both are playing best response):

R1 \ R2	Left	Right
Left	(0, 0)	(5, 5)
Right	(5, 5)	(0, 0)

Nash Equilibria:

1. **(Left, Right) = (5, 5)** — R1 goes Left, R2 goes Right (both underlined)
2. **(Right, Left) = (5, 5)** — R1 goes Right, R2 goes Left (both underlined)

Interpretation: The robots successfully coordinate when they choose *opposite* lanes, avoiding collision. This is an **anti-coordination game** — both equilibria give payoff 5 to each player.
Grading: 1pt for correct best response analysis, 1pt for identifying both NE.

Question III.3: Project Code — Logic Agent (2 pts, 8 min)

Given: Knowledge Base code from the RoboMind project's `logic_agent.py`:

```python
def get_safe_moves(self, percept):
    # Add current sensor readings to KB
    for direction in ["North", "East", "South", "West"]:
        if percept[direction] == "wall":
            self.kb.tell(f"Wall({direction})")
        else:
            self.kb.tell(f"Free({direction})")

    # Apply rule: Free(X) => Safe(X)
    self.kb.infer()

    # Find all safe moves
    safe_moves = []
    for action in ["North", "East", "South", "West"]:
        if self.kb.ask(f"Safe({action})"):
            safe_moves.append(action)

    return safe_moves
```

Scenario: The robot's sensors detect: wall to North, wall to East, free to South, free to West.
Tasks:

(a) [1 pt] What is the purpose of `kb.tell()` vs `kb.ask()`? How do they differ?

(b) [1 pt] Trace through the for-loop that builds `safe_moves`. For each direction, show whether `kb.ask()` returns True or False, and list the final contents of `safe_moves`.

(a) kb.tell() vs kb.ask() [1 pt]:
kb.tell(sentence):

- **Purpose:** Add new knowledge to the knowledge base
- Asserts a fact or rule as TRUE
- Example: `kb.tell("Wall(North)")` adds the fact that North has a wall

kb.ask(query):

- **Purpose:** Query the knowledge base to check if something is true
- Returns True/False based on logical inference
- Example: `kb.ask("Safe(South)")` checks if South is safe

tell() = ADD knowledge, ask() = QUERY knowledge

Grading: 0.5pt for each correct explanation.

(b) Code Trace through safe_moves loop [1 pt]:
After first loop and kb.infer():

- KB facts: `Wall(North)`, `Wall(East)`, `Free(South)`, `Free(West)`
- Rule applied: `Free(X)` ⇒ `Safe(X)`
- Inferred: `Safe(South)`, `Safe(West)`

Trace through for-loop (line by line):

Iteration	action	`kb.ask(f"Safe({action})")`	safe_moves
1	"North"	`ask("Safe(North)")` → **False**	[]
2	"East"	`ask("Safe(East)")` → **False**	[]
3	"South"	`ask("Safe(South)")` → **True**	["South"]
4	"West"	`ask("Safe(West)")` → **True**	["South", "West"]

Return value:

`safe_moves` = ["South","West"]

Explanation: The rule `Free(X)` ⇒ `Safe(X)` only fires for South and West because they are the only directions marked as Free. North and East have walls, so no rule derives Safe for them.
Grading: 1pt for correct trace showing South and West with reasoning.